Press-A-Button Voice Typing on Ubuntu

KDE Plasma / Wayland Setup Guide

Using **Handy** + **Input Remapper** + a USB macro button
for hands-free local speech-to-text

Daniel Rosehill
March 2026

Contents

1 Introduction . . . 3
1.1 Why This Guide Exists . . . 3
1.2 Architecture Overview . . . 3
2 Hardware . . . 4
2.1 Single USB Macro Button . . . 4
2.2 USB Foot Pedal . . . 4
2.3 USB Macro Pad . . . 5
3 Installing Handy . . . 7
4 Configuring Handy . . . 8
4.1 General Settings . . . 8
4.2 Choosing a Transcription Model . . . 8
4.3 Advanced Settings . . . 11
5 Installing Input Remapper . . . 14
6 Configuring Input Remapper . . . 15
6.1 Step 1: Select Your Device . . . 15
6.2 Step 2: Create a Preset . . . 15
6.3 Step 3: Map the Button . . . 15
7 Single Button vs Multi-Button Devices . . . 17
7.1 Single Button (Toggle Mode) . . . 17
7.2 Multi-Button Macro Pad . . . 17
8 The Complete Workflow . . . 18
9 Wayland Troubleshooting . . . 19
10 GPU Acceleration (AMD) . . . 19
10.1 Inference Benchmarks . . . 19
10.2 Viewing Your Own Benchmarks . . . 20
11 System Requirements . . . 21
12 Software References . . . 21

1 Introduction

This guide covers setting up one-press voice typing on Ubuntu 25.10 with KDE Plasma on Wayland using three components:

- **Handy** — a local speech-to-text tool
- **Input Remapper** — a GUI tool for remapping input devices on Linux
- A USB macro button, foot pedal, or macro pad

The end result: press a physical button, speak, and your words appear as typed text wherever your cursor is.

1.1 Why This Guide Exists

Handy works well out of the box on X11, but getting it running smoothly on **KDE Plasma + Wayland** required several non-obvious workarounds:

1. The **overlay had to be disabled** — with it enabled, typed output wouldn't reach the active window.
2. **ydotool had to be manually selected** as the typing tool — the default input method doesn't work on Wayland.
3. **F13 would have been the ideal trigger key** (unlikely to conflict with anything), but Handy currently only supports a limited set of shortcut keys. `Pause` was used as a workaround.

If you're on X11, you may not need all of these steps. On Wayland (the default on modern Ubuntu + KDE), this guide should save you significant troubleshooting.

1.2 Architecture Overview

Component	Role
USB HID Device	Physical trigger — button, foot pedal, or macro pad
Input Remapper	Maps device button press to `KEY_PAUSE` keyboard shortcut
Handy	Listens for shortcut, records audio, transcribes, and types the result via ydotool
ydotool	Wayland-compatible virtual keyboard for typing output into the active window

Table 1: System components and their roles

2 Hardware

Any programmable USB HID device will work. The device just needs to show up as a HID input device on Linux — no special drivers required.

2.1 Single USB Macro Button

The simplest option. A single large button that sends one keycode. Works well with Handy's toggle-to-transcribe shortcut. Available cheaply on Amazon and AliExpress.

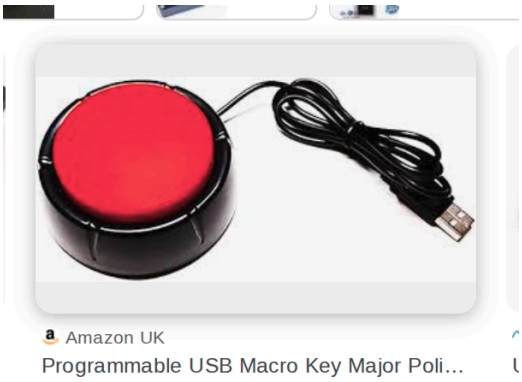

Figure 1: USB macro button available on Amazon

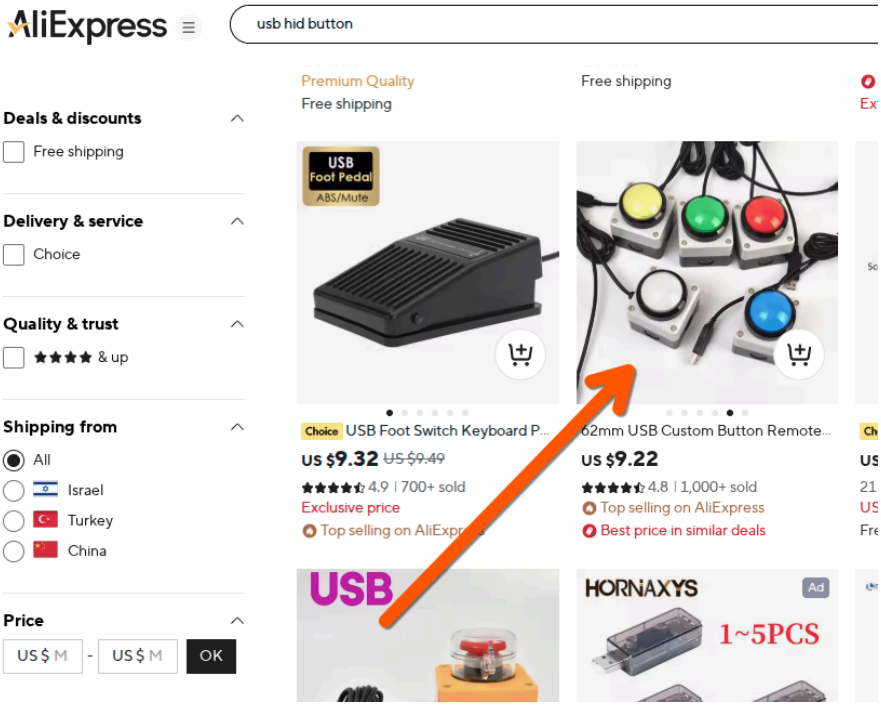

Figure 2: USB macro button listings on AliExpress

2.2 USB Foot Pedal

Hands-free operation with 1–3 keys. Great if you're typing and dictating simultaneously.

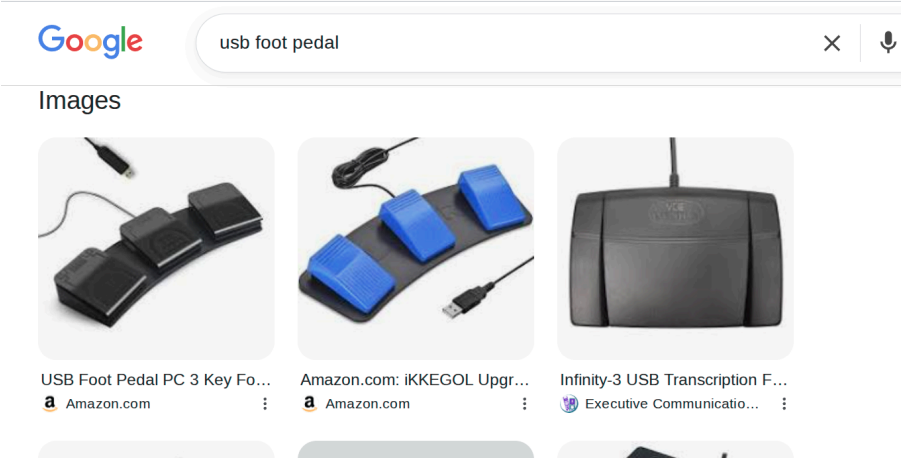

Figure 3: USB foot pedals — various models available

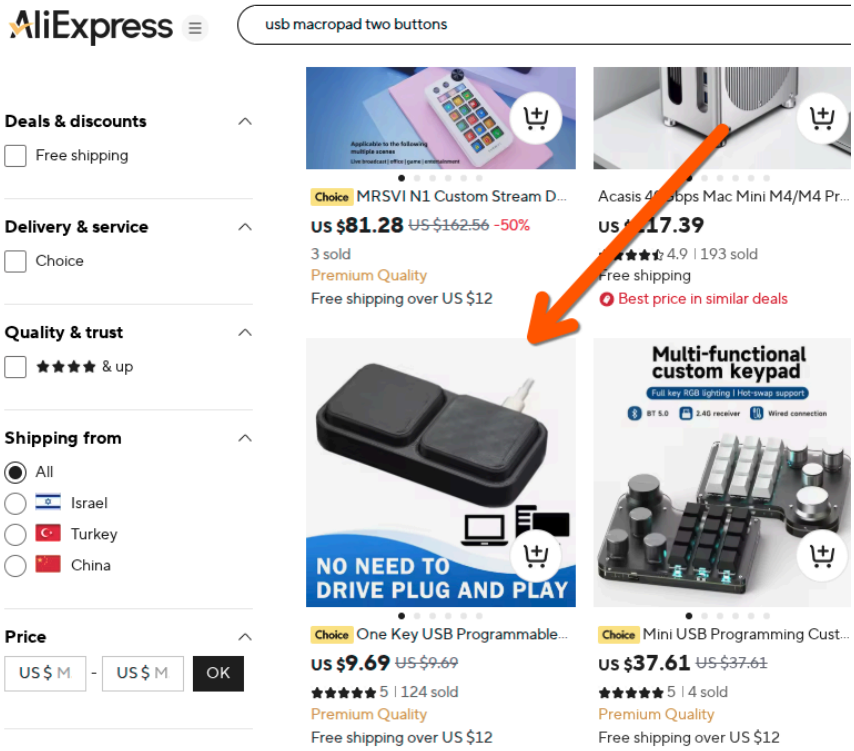

Figure 4: USB foot pedal listings on AliExpress

2.3 USB Macro Pad

The most flexible option. With multiple buttons you can assign separate shortcuts for start, stop, and push-to-talk rather than relying on a single toggle.

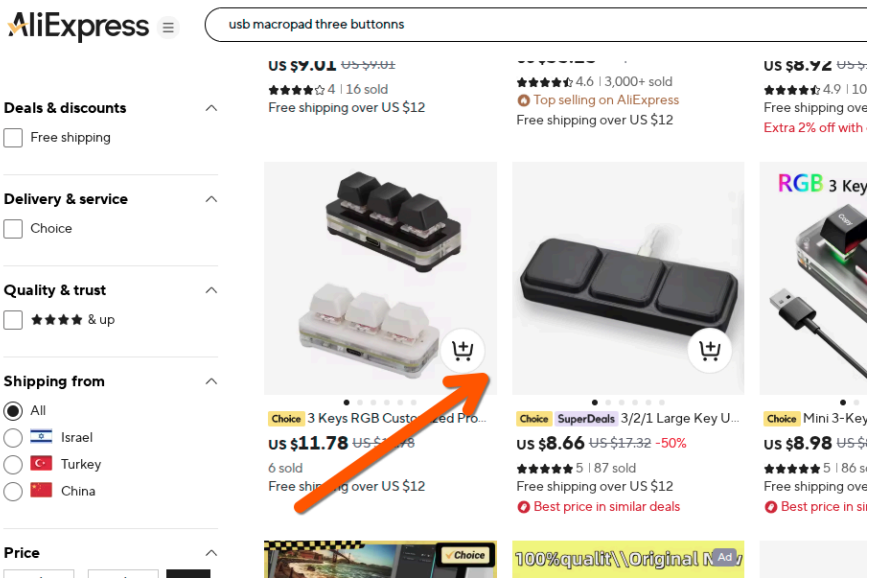

Figure 5: USB macro pads on AliExpress — multi-button options

3 Installing Handy

Install Handy from its GitHub repository or package. It runs as a background app with a system tray icon.

Repository: `https://github.com/pokey/handy`

4 Configuring Handy

4.1 General Settings

Configure the following in the **General** tab:

- **Transcribe Shortcut**: Set to `Pause` (or any key you prefer)
- **Microphone**: Select your preferred mic
- **Audio Feedback**: Enable for an audible cue when transcription starts/stops

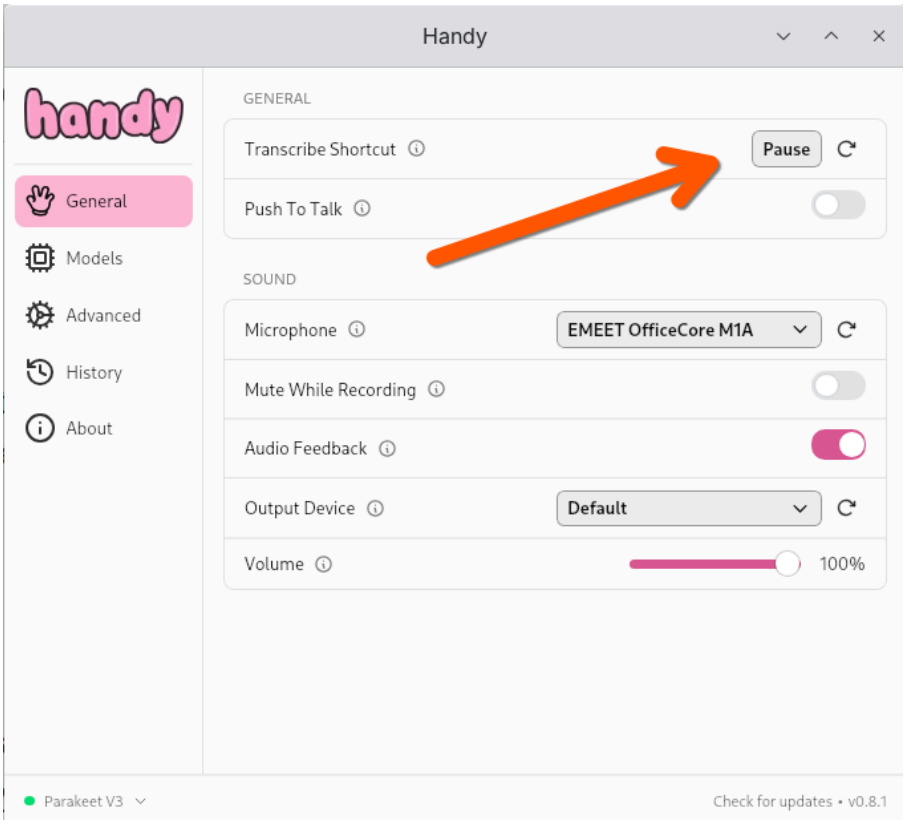

Figure 6: General settings — Transcribe Shortcut set to Pause

4.2 Choosing a Transcription Model

Select a model from the **Models** tab. **Parakeet V3** offers a good balance of accuracy and speed with multi-language support.

Model	Speed	Accuracy	Languages
Parakeet V3 *(recommended)*	Fast	High	Multi-language
Moonshine Base	Very fast	Good	English only
Moonshine V2 Medium	Fast	High	English only
Whisper Small	Fast	Fair	Multi-language
Whisper Medium	Medium	Good	Multi-language
Whisper Large	Slow	High	Multi-language
Whisper Turbo	Fast	Good	Multi-language
SenseVoice	Very fast	Good	CJK + English
Canary 180M Flash	Very fast	Good	Multi-language
Canary 1B v2	Medium	High	25 languages

Table 2: Available transcription models

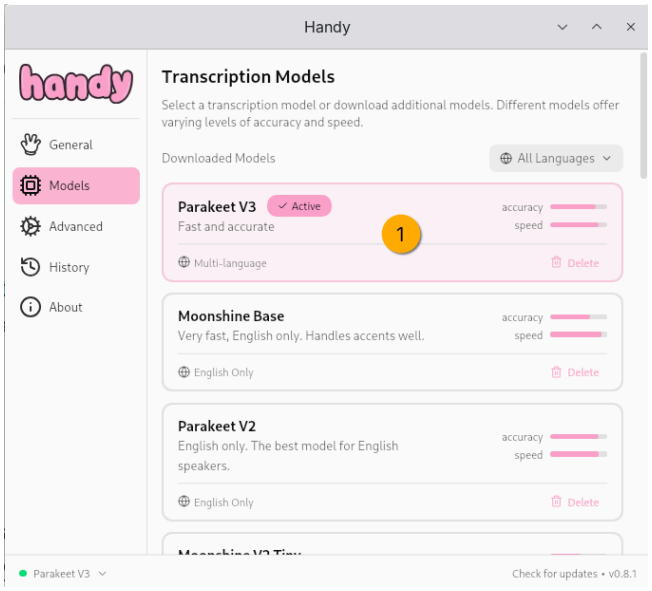

Figure 7: Parakeet V3 selected as the active model

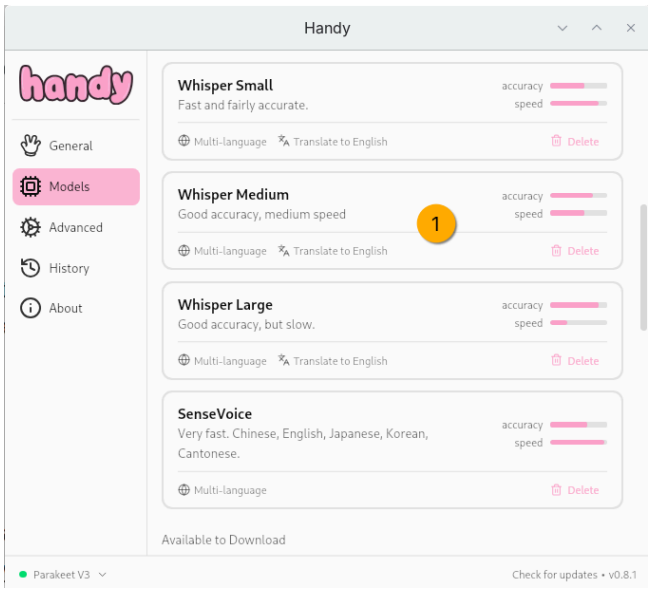

Figure 8: Additional models — Whisper variants and SenseVoice

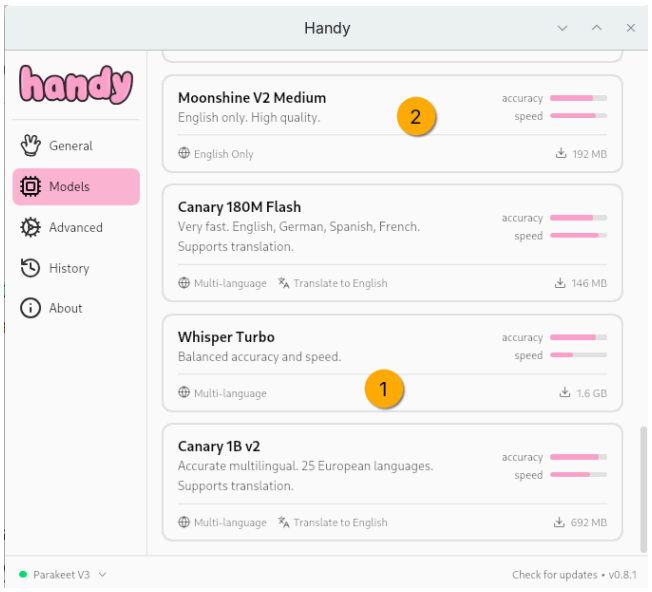

Figure 9: More models — Moonshine V2, Canary, Whisper Turbo

4.3 Advanced Settings

These settings are critical for Wayland compatibility.

Setting	Value	Reason
Start Hidden	On	Runs quietly in the background
Launch on Startup	On	Always available
Show Tray Icon	On	Easy access to settings
Overlay Position	**None**	**Required on Wayland** — overlay prevents typed output reaching the active window
Unload Model	**Never**	Keeps the model in RAM for instant response
Paste Method	**Direct**	Types directly into the active window
Typing Tool	**ydotool**	**Required on Wayland** — the default typing method does not work
Clipboard Handling	Don't Modify	Avoids overwriting clipboard contents
History Limit	1 entry	Saves disk space
Auto-Delete Recordings	Keep latest 1	Privacy-friendly

Table 3: Recommended Advanced settings (bold = Wayland-critical)

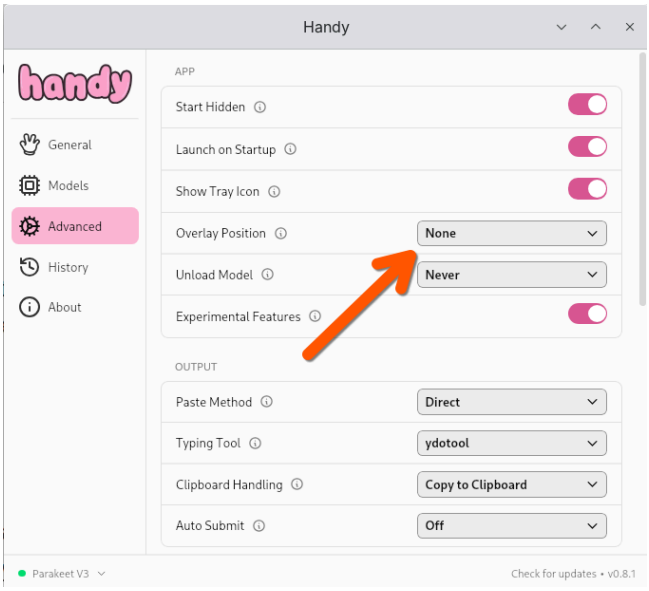

Figure 10: Advanced settings overview — Overlay Position set to None, Typing Tool set to ydotool

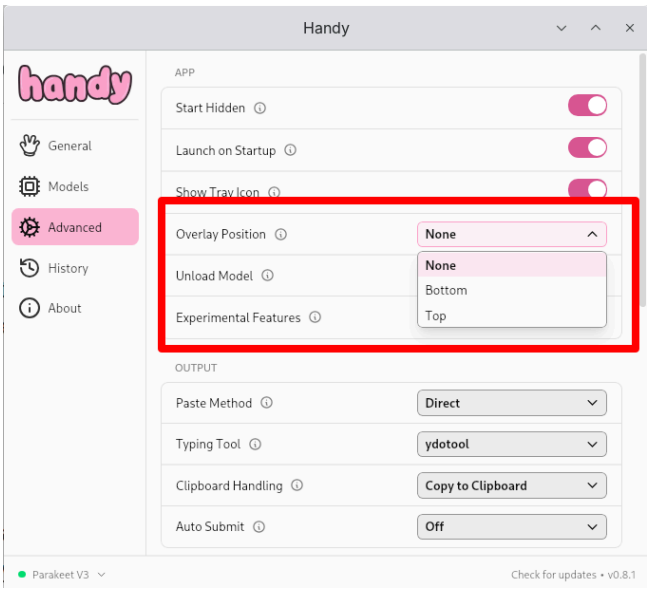

Figure 11: Overlay Position dropdown — select None

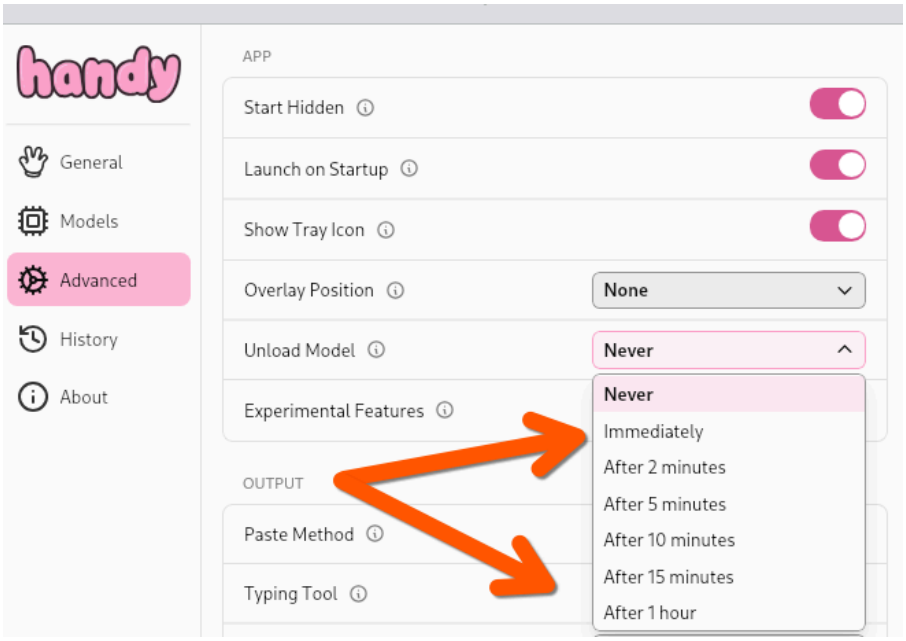

Figure 12: Unload Model dropdown — set to Never

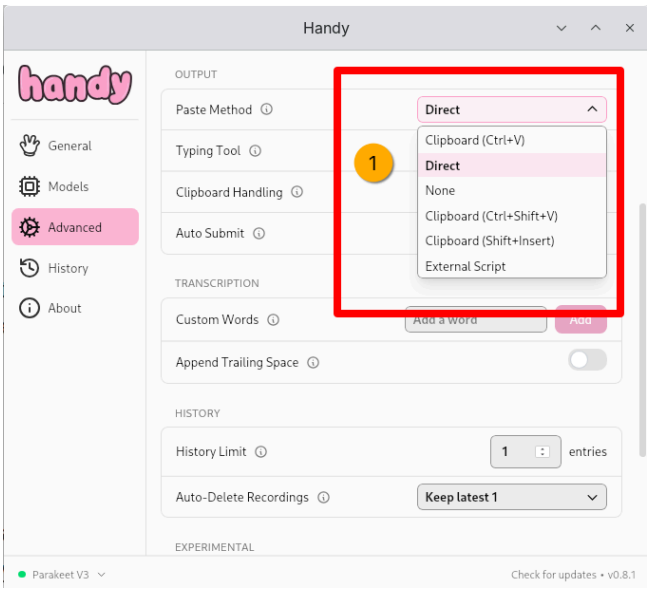

Figure 13: Paste Method dropdown — select Direct

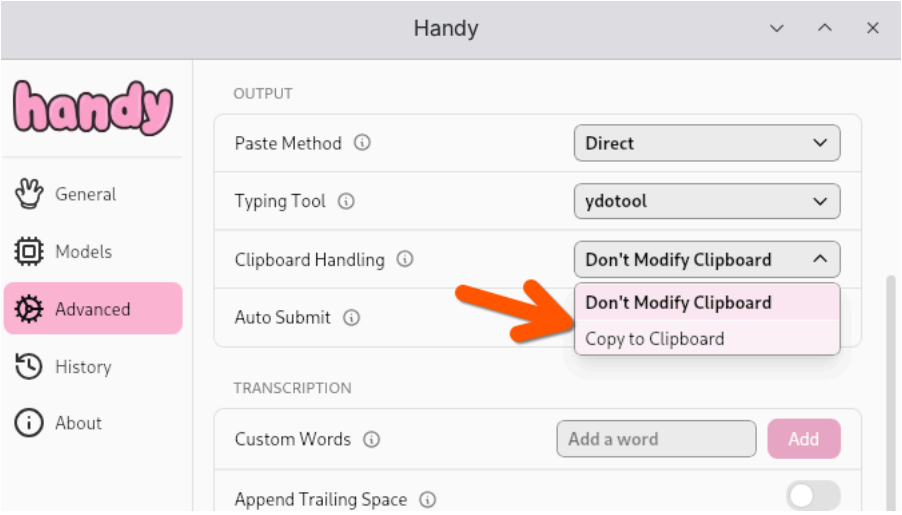

Figure 14: Clipboard Handling — select Don’t Modify Clipboard

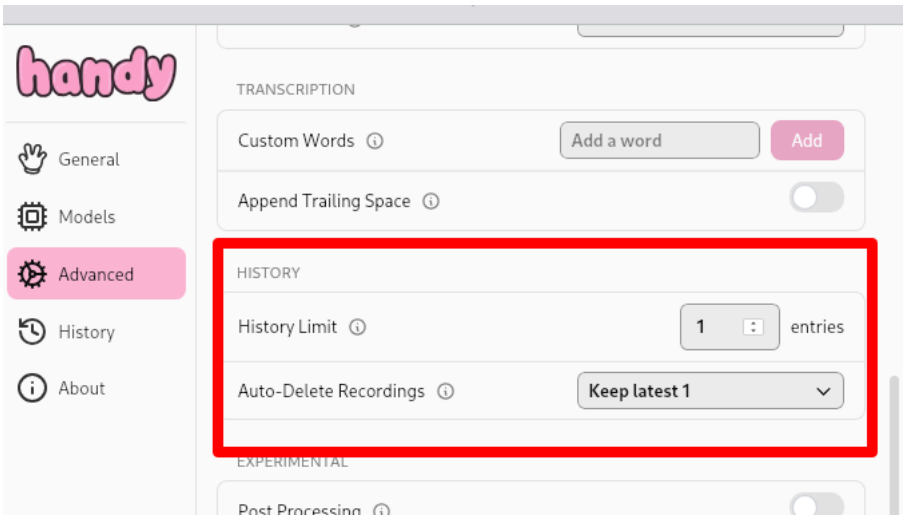

Figure 15: History settings — limit to 1 entry, keep latest 1 recording

5 Installing Input Remapper

Install via apt:

```
sudo apt install input-remapper
```

Input Remapper provides a GUI for remapping any input device on Linux. It runs as a system service and persists mappings across reboots.

6 Configuring Input Remapper

6.1 Step 1: Select Your Device

Open Input Remapper and find your USB device in the **Devices** tab. It will appear by its HID identifier (e.g., `HID 5131:2019`).

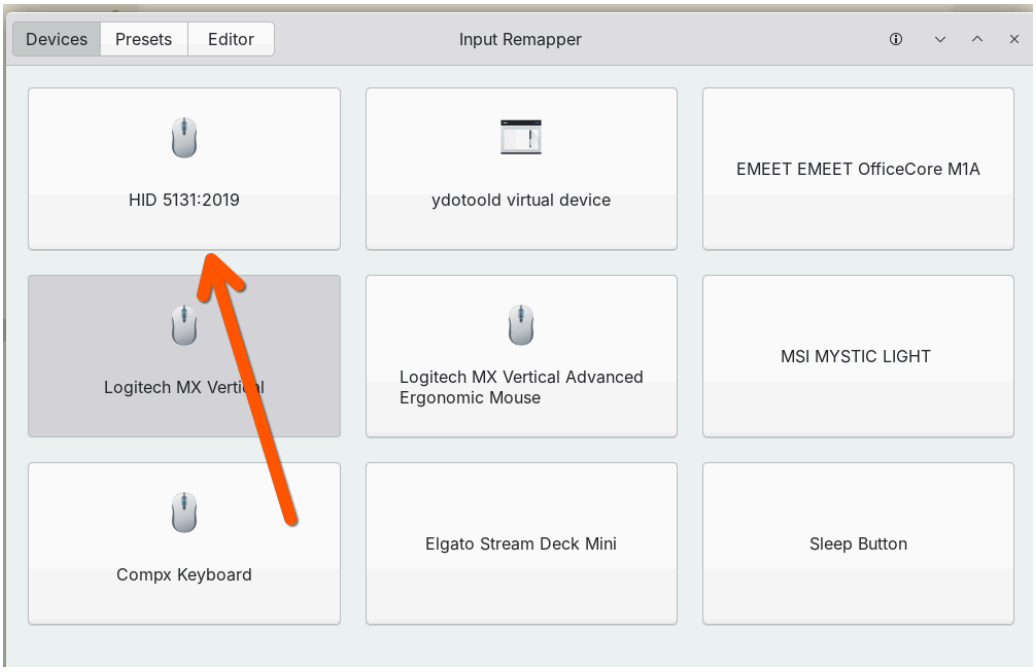

Figure 16: Devices tab — select your USB HID device

6.2 Step 2: Create a Preset

Go to the **Presets** tab and create a new preset (e.g., “USB Voice Typing Trigger”).

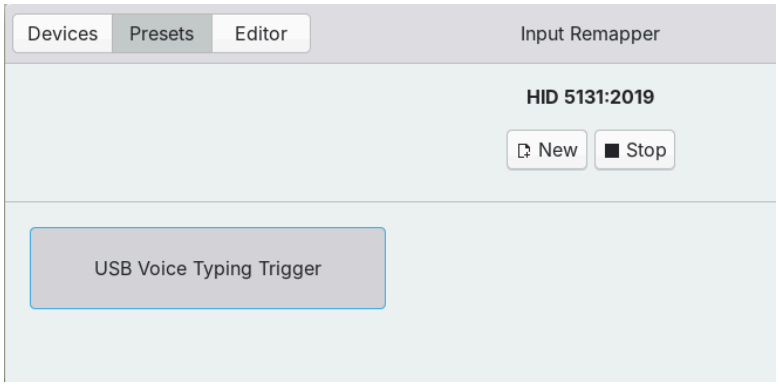

Figure 17: Presets tab — USB Voice Typing Trigger preset created

6.3 Step 3: Map the Button

In the **Editor** tab:

1. Click **Record** and press your USB button to capture the input
2. Set the output type to **Key or Macro**
3. Set the target to **keyboard**
4. Map it to `KEY_PAUSE`

5. Enable **Autoload** so the mapping persists across reboots
6. Click **Apply**

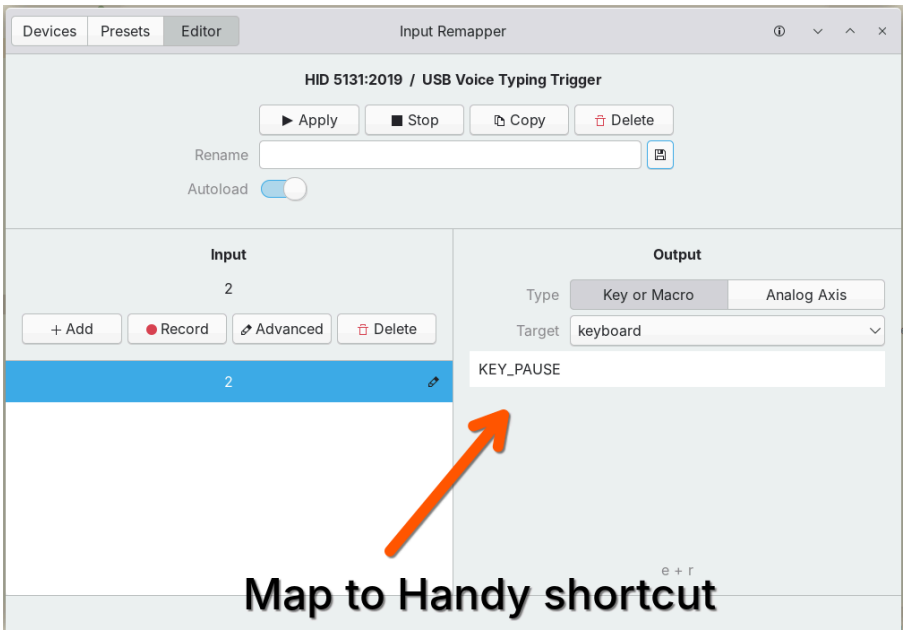

Figure 18: Editor tab — button mapped to KEY_PAUSE with Autoload enabled

Note: Ideally you'd use `F13` or another rarely-used key, but Handy currently only supports a limited set of shortcut keys. `Pause` is a good choice since it's rarely used by other applications.

7 Single Button vs Multi-Button Devices

7.1 Single Button (Toggle Mode)

With a single button, you use Handy's toggle mode: press once to start transcribing, press again to stop. Simple and effective.

7.2 Multi-Button Macro Pad

With a macro pad (3+ buttons), you can assign separate shortcuts for:

- **Start** transcription
- **Stop** transcription
- **Push-to-talk** (hold to record, release to stop)

This gives finer control and avoids accidentally toggling into the wrong state.

8 The Complete Workflow

Once everything is configured, the workflow is seamless:

1. Handy launches silently at boot and loads the transcription model into memory
2. Input Remapper loads at boot and maps your USB device
3. Place your cursor in any text field
4. Press your button and speak
5. Text appears where your cursor is

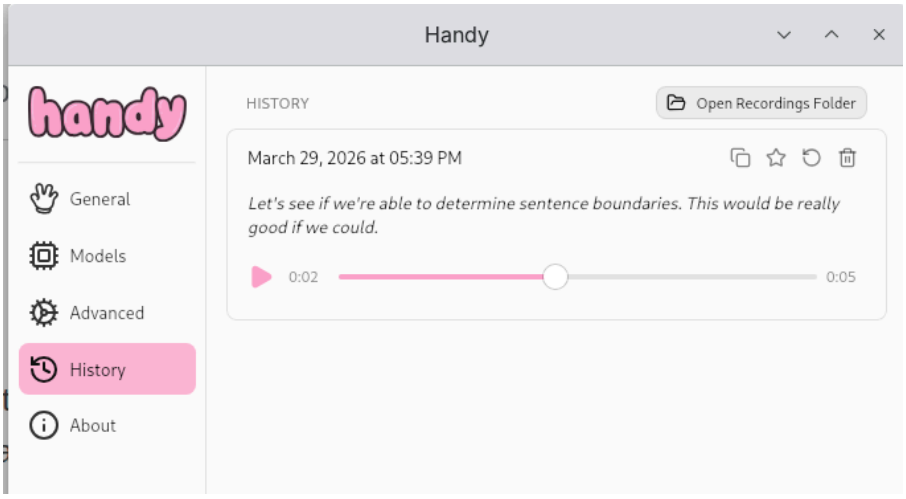

Figure 19: Example transcription result in Handy's History view

9 Wayland Troubleshooting

If transcription runs but no text appears in your application, check the following:

#	Check	Action
1	Overlay disabled	Set Overlay Position to `None` in Advanced settings
2	Typing Tool	Set to `ydotool` in Advanced settings
3	ydotool installed	Run `sudo apt install ydotool` and verify `ydotoold` is active
4	Paste Method	Set to `Direct` in Advanced settings

Table 4: Wayland troubleshooting checklist

10 GPU Acceleration (AMD)

This setup was tested on an **AMD Radeon RX 7800 XT** (Navi 32, 12 GB VRAM) with ROCm. Handy uses ONNX Runtime for inference and automatically detects the AMD GPU — no manual configuration is needed.

The `ort_accelerator` setting defaults to `auto`, and the log confirms GPU selection:

```
Auto-selected GPU device 0 'AMD Radeon RX 7700 XT (RADV NAVI32)'
(Dedicated, 12288 MB VRAM)
```

10.1 Inference Benchmarks

Benchmarks from Handy's debug log using **Parakeet V3 (INT8)** on the RX 7800 XT:

Recording	Inference	RTF	Transcribed Text
4 sec	574 ms	0.14x	*"Okay, we're mapping out to the pause shortcuts here."*
6 sec	912 ms	0.15x	*"Let's see if we're able to determine sentence boundaries…"*
16 sec	1,695 ms	0.11x	*"It takes a few steps, but being able to enter text seamlessly…"*
4 sec	554 ms	0.14x	*"This text was written."*
24 sec	1,603 ms	0.07x	*"This text was written with parakeet, and the objective…"* (long paragraph)

Table 5: Inference benchmarks — Parakeet V3 (INT8) on AMD RX 7800 XT

Model load time: 1,060–1,870 ms (first load is slower).

A real-time factor (RTF) below 1.0 means inference is faster than real-time. Parakeet V3 on this GPU consistently achieves **0.07–0.15x RTF**, meaning transcription completes in roughly 1/10th the time of the recording.

10.2 Viewing Your Own Benchmarks

Check Handy's log for transcription timing:

```
grep "Transcription completed" \
  ~/.local/share/com.pais.handy/logs/handy.log
```

11 System Requirements

- Ubuntu 25.10 (or similar) with KDE Plasma on Wayland
- A USB HID macro button, macro pad, or foot pedal
- Sufficient RAM for the transcription model (varies by model size)
- **GPU (optional but recommended)**: AMD GPU with ROCm support for accelerated inference. Tested on RX 7800 XT. CPU-only inference also works but will be slower.

12 Software References

Software	Description & Link
Handy	`https://github.com/pokey/handy` — Local speech-to-text with direct typing output
Input Remapper	`https://github.com/sezanzeb/input-remapper` — GUI tool for remapping input devices on Linux
ydotool	Wayland-compatible virtual keyboard tool (used by Handy for typing output)

Table 6: Software used in this setup